Floating Point

15-213/18-213/15-513: Introduction to Computer Systems
4th Lecture, Sept. 7, 2017

Today's Instructor:

Phil Gibbons

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

科学计数法(Scientific Notation)与浮点数

Example:

mantissa (尾数) *exponent(阶码、指数)*

$$6.02 \times 10^{21}$$

decimal point *radix (base，基)*

- **Normalized form（规格化形式）：小数点前只有一位非0数**
- **同一个数有多种表示形式。例：对于数 1/1,000,000,000**
  - **Normalized (唯一的规格化形式): 1.0×10^{-9}**
  - **Unnormalized（非规格化形式不唯一）: 0.1×10^{-8}, 10.0×10^{-10}**

for Binary Numbers:

mantissa（尾数） *exponent（指数）*

$$0.101_{two} \times 2^{-10}$$

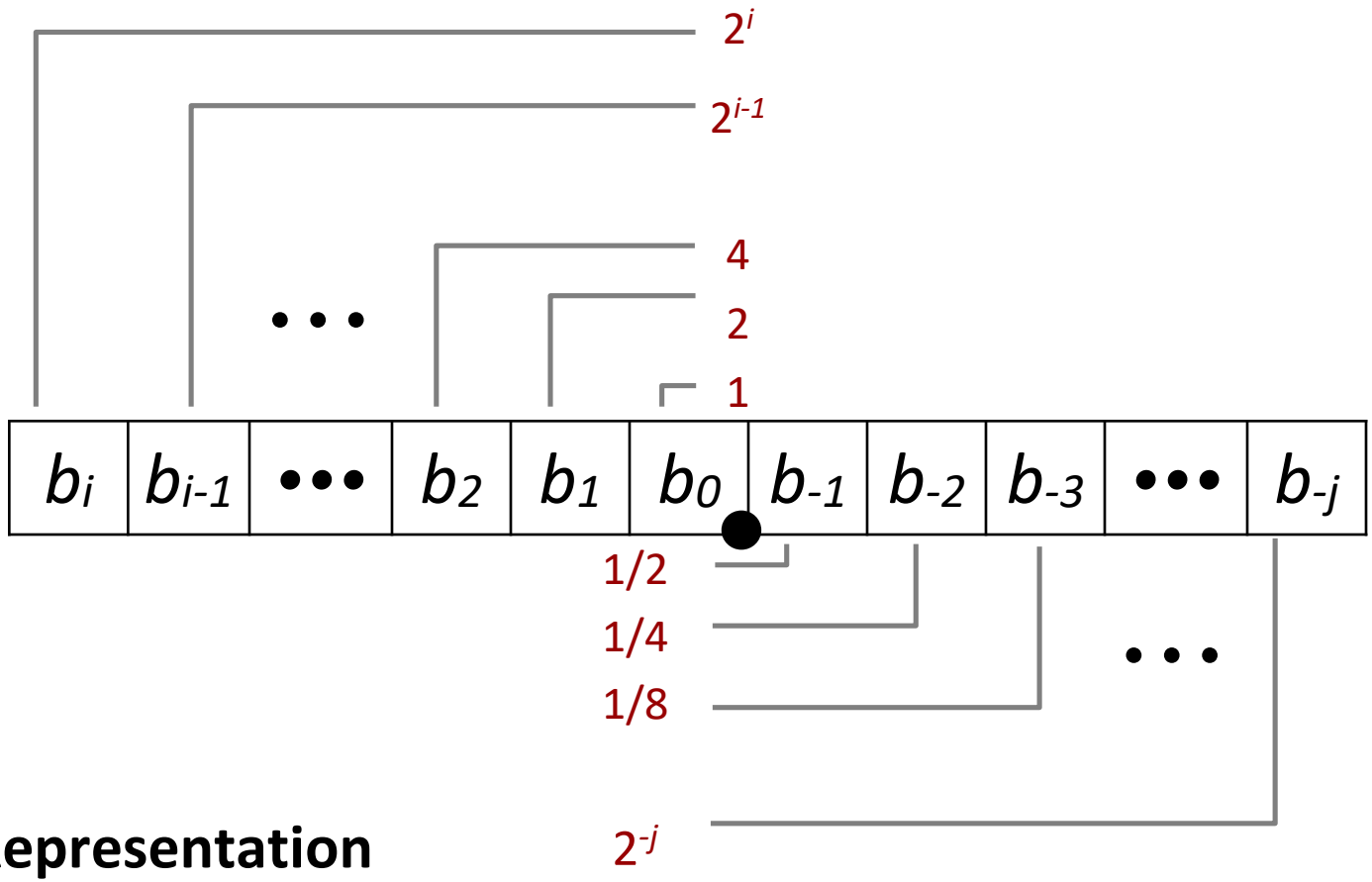
binary point *基为2*

只要对尾数和指数分别编码，就可表示一个浮点数（即：实数）

Fractional binary numbers

- What is 1011.101_2?

Fractional Binary Numbers



- **Representation**
  - Bits to right of "binary point" represent fractional powers of 2
  - Represents rational number: $$\sum_{k=-j}^{i} b_k \times 2^k$$

Fractional Binary Numbers: Examples

- **Value** **Representation**

5 3/4	= 23/4	101.11_2	= 4 + 1 + 1/2 + 1/4
2 7/8	= 23/8	10.111_2	= 2 + 1/2 + 1/4 + 1/8
1 7/16	= 23/16	1.0111_2	= 1 + 1/4 + 1/8 + 1/16

- **Observations**
  - Divide by 2 by shifting right (unsigned)
  - Multiply by 2 by shifting left
  - Numbers of form $0.111111\ldots_2$ are just below 1.0
    - $1/2 + 1/4 + 1/8 + \ldots + 1/2^i + \ldots \rightarrow 1.0$
    - Use notation $1.0 - \varepsilon$

Representable Numbers

- **Limitation #1**
  - Can only exactly represent numbers of the form $x/2^k$
    - Other rational numbers have repeating bit representations
  - Value Representation
    - 1/3 `0.0101010101[01]`$\ldots_2$
    - 1/5 `0.001100110011[0011]`$\ldots_2$
    - 1/10 `0.0001100110011[0011]`$\ldots_2$

- **Limitation #2**
  - Just one setting of binary point within the *w* bits
    - Limited range of numbers (very small values? very large?)

浮点数的表示

- Normal format（规格化数形式）：

  +/-1.xxxxxxxxxxx × $R^{Exponent}$

- 32-bit 规格化数：

31		0
S	Exponent	Significand
1 bit	? bits	? bits

S 是符号位（Sign）

Exponent用移码（增码）来表示

Significand 表示 xxxxxxxxxxxxx，尾数部分

(基可以是 2/ 4 / 8 / 16，约定信息，无需显式表示)

- 早期的计算机，各自定义自己的浮点数格式

  问题：浮点数表示不统一会带来什么问题？

Today: Floating Point

- Background: Fractional binary numbers
- **IEEE floating point standard: Definition**
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

“Father” of the IEEE 754 standard

直到80年代初，各个机器内部的浮点数表示格式还没有统一
因而相互不兼容，机器之间传送数据时，带来麻烦

1970年代后期，IEEE成立委员会着手制定浮点数标准

1985年完成浮点数标准IEEE 754的制定

This standard was primarily the work of one person, UC Berkeley math professor William Kahan.

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html

Prof. William Kahan

IEEE Floating Point

- **IEEE Standard 754**
  - Established in 1985 as uniform standard for floating point arithmetic
    - Before that, many idiosyncratic formats
  - Supported by all major CPUs
  - Some CPUs don't implement IEEE 754 in full
    e.g., early GPUs, Cell BE processor

- **Driven by numerical concerns**
  - Nice standards for rounding, overflow, underflow
  - Hard to make fast in hardware
    - Numerical analysts predominated over hardware designers
      in defining standard

Floating Point Representation

Example:
$15213_{10} = (-1)^0 \times 1.1101101101101_2 \times 2^{13}$

- **Numerical Form:**

$$(-1)^s\, M\ \ 2^E$$

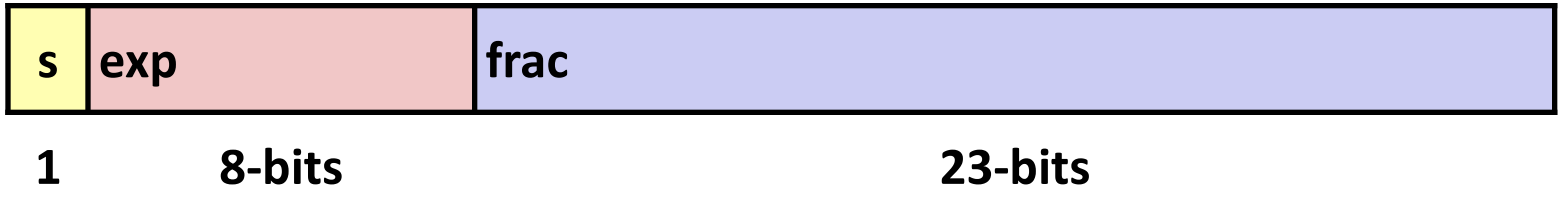
  - **Sign bit *s*** determines whether number is negative or positive
  - **Significand *M*** normally a fractional value in range [1.0,2.0).
  - **Exponent *E*** weights value by power of two

- **Encoding**
  - MSB s is sign bit ***s***
  - **exp** field encodes ***E*** (but is not equal to E)
  - **frac** field encodes ***M*** (but is not equal to M)

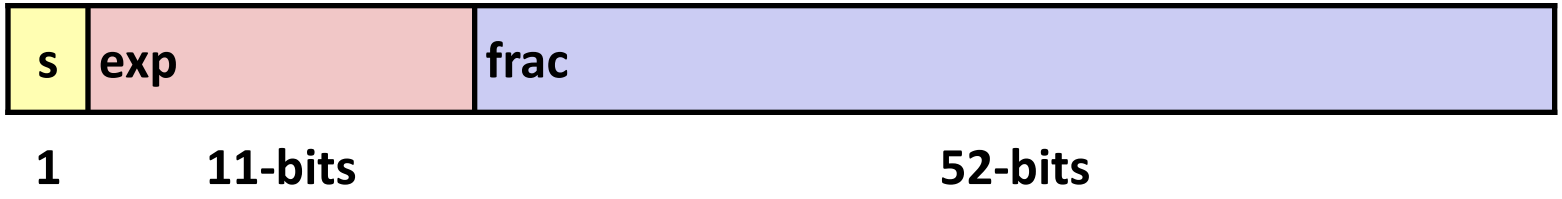
s	exp	frac

Precision options

- **Single precision: 32 bits**
  ≈ 7 decimal digits, $10^{\pm 38}$

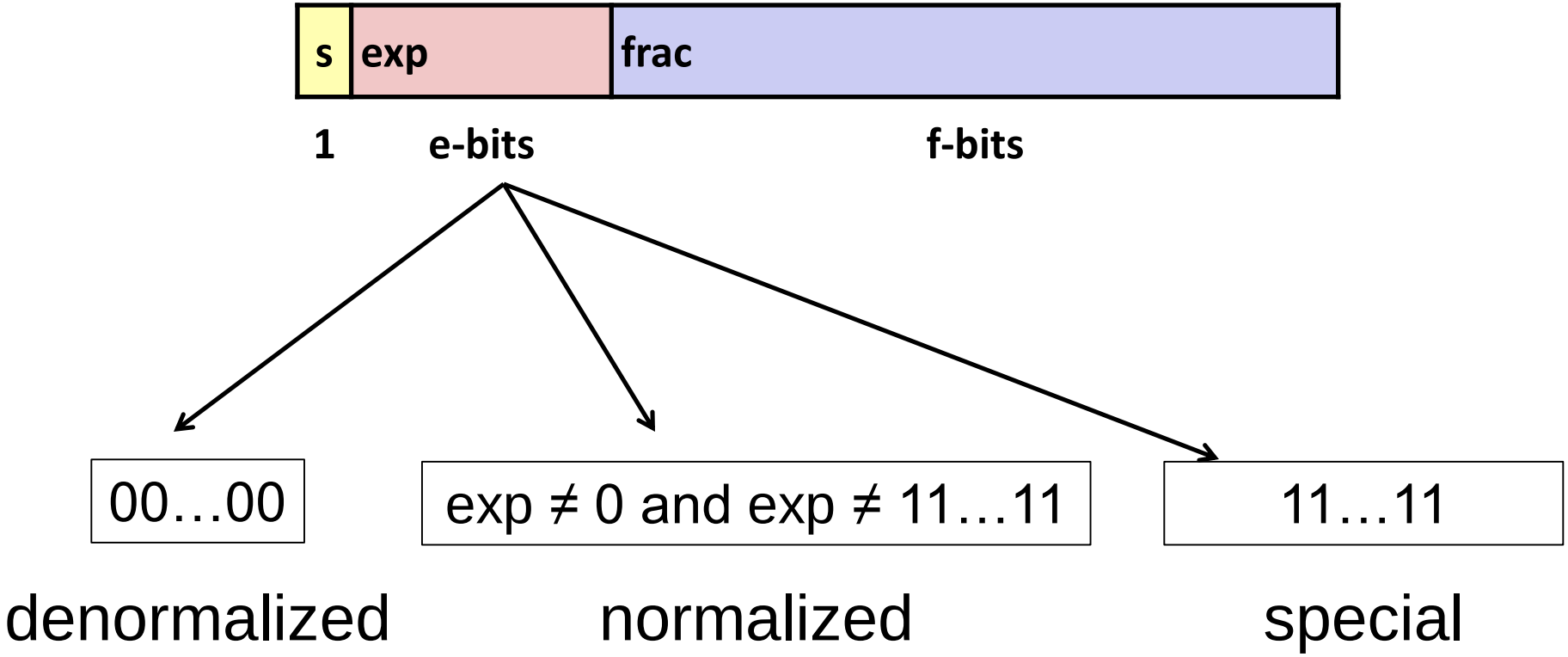


- **Double precision: 64 bits**
  ≈ 16 decimal digits, $10^{\pm 308}$



- **Other formats: half precision, quad precision**

Three "kinds" of floating point numbers



"Normalized" Values

$v = (-1)^s \, M \, 2^E$

- **When: `exp` ≠ 000…0 and `exp` ≠ 111…1**

- **Exponent coded as a *biased* value: E = `exp` – *Bias***
  - *exp*: unsigned value of exp field
  - *Bias* = $2^{k-1} - 1$, where *k* is number of exponent bits
    - Single precision: 127 (**`exp`**: 1…254, E: -126…127)
    - Double precision: 1023 (**`exp`**: 1…2046, E: -1022…1023)

- **Significand coded with implied leading 1: M =** $1.xxx\ldots x_2$
  - xxx…x: bits of frac field
  - Minimum when **`frac`**=000…0 (M = 1.0)
  - Maximum when **`frac`**=111…1 (M = 2.0 – ε)
  - Get extra leading bit for "free"

Normalized Encoding Example

$$v = (-1)^s \, M \, 2^E$$
$$E = \texttt{exp} - Bias$$

- **Value:** `float F = 15213.0;`
  - $15213_{10} = 11101101101101_2$
    $= 1.1101101101101_2 \times 2^{13}$

- **Significand**

  $M = 1.\underline{1101101101101}_2$

  $\texttt{frac} = \underline{1101101101101}0000000000_2$

- **Exponent**

  $E = 13$

  $Bias = 127$

  $\texttt{exp} = 140 = 10001100_2$

- **Result:**

0	10001100	11011011011010000000000
s	exp	frac

IEEE 754标准

规定：小数点前总是“1”，故可隐含表示。

注意：和前面例子规定不一样，这里更合理！

规格化数：+/-1.xxxxxxxxxx$_{two}$ x $2^{Exponent}$

Single Precision：

S	Exponent	Significand
1 bit	8 bits	23 bits

- Sign bit: 1 表示negative ; 0表示 positive
- Exponent（阶码 / 指数）：

  全0和全1用来表示特殊值！

  - SP规格化数阶码范围为0000 0001 (-126) ~ 1111 1110 (127)
  - bias为127 (single), 1023 (double)

  为什么用127？若用128，则阶码范围为多少？

- Significand（尾数）：
  - 规格化尾数最高位总是1，所以隐含表示，省1位
  - 1 + 23 bits （ single），1 + 52 bits （double）

SP: $(-1)^S$ x (1 + Significand) x $2^{(Exponent-127)}$

DP: $(-1)^S$ x (1 + Significand) x $2^{(Exponent-1023)}$

0000 0001 (-127) ~ 1111 1110 (126)

Ex: Converting Binary FP to Decimal

BEE00000H is the hex. Rep. Of an IEEE 754 SP FP number

1	011 11101	110 0000 0000 0000 0000 0000

$$(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-127)}$$

- **Sign: 1 => negative**
- **Exponent:**
  - **$0111\ 1101_{two} = 125_{ten}$**
  - **Bias adjustment: 125 - 127 = -2**
- **Significand:**

  $1 + 1\times2^{-1} + 1\times2^{-2} + 0\times2^{-3} + 0\times2^{-4} + 0\times2^{-5} + \ldots$

  $=1+2^{-1} +2^{-2} = 1+0.5 +0.25 = 1.75$
- **Represents: $-1.75_{ten}\times2^{-2} = -0.4375$**

Ex: Converting Decimal to FP

-12.75

1. Denormalize: -12.75
2. Convert integer part:

   $12 = 8 + 4 = 1100_2$
3. Convert fractional part:

   $.75 = .5 + .25 = .11_2$
4. Put parts together and normalize:

   $1100.11 = 1.10011 \times 2^3$
5. Convert exponent: $127 + 3 = 128 + 2 = 1000\ 0010_2$

1	1000 0010	100 1100 0000 0000 0000 0000

The Hex rep. is C14C0000H

Normalized numbers（规格化数）

前面的定义都是针对规格化数（normalized form）

How about other patterns?

Exponent	Significand	Object
1-254	anything implicit leading 1	Norms
0	0	?
0	nonzero	?
255	0	?
255	nonzero	?

Denormalized Values

$$v = (-1)^s \, M \, 2^E$$
$$E = 1 - Bias$$

- **Condition:** exp = 000…0

- **Exponent value: *E* = 1 – Bias (instead of `exp` – *Bias*)**
- **Significand coded with implied leading 0: *M* = 0.xxx…x$_2$**
  - `xxx…x`: bits of `frac`
- **Cases**
  - `exp` = 000…0, `frac` = 000…0
    - Represents zero value
    - Note distinct values: +0 and –0 (表示不同含义)
  - `exp` = 000…0, `frac` ≠ 000…0
    - Numbers closest to 0.0
    - Equispaced（可能的数值分布均匀地接近0）

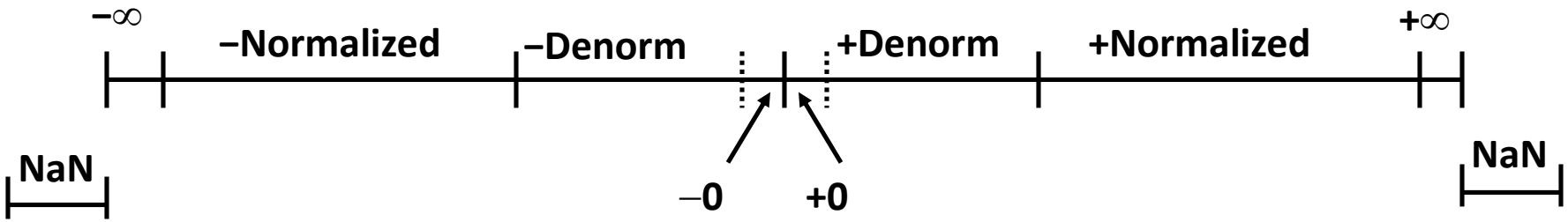
Special Values

- **Condition: `exp` = `111…1`**

- **Case: `exp` = `111…1`, `frac` = `000…0`**
  - **Represents value ∞ (infinity)**
  - Operation that overflows
  - Both positive and negative
  - E.g., 1.0/0.0 = −1.0/−0.0 = +∞, 1.0/−0.0 = −∞

- **Case: `exp` = `111…1`, `frac` ≠ `000…0`**
  - **Not-a-Number (NaN)**
  - Represents case when no numeric value can be determined
  - E.g., sqrt(–1), ∞ − ∞, ∞ × 0

Visualization: Floating Point Encodings



C float Decoding Example

$$v = (-1)^s \, M \, 2^E$$
$$E = \texttt{exp} - Bias$$

$Bias = 2^{k-1} - 1 = 127$

float: `0xC0A00000`

binary: _____ _____ _____ _____ _____ _____ _____ _____

E =

S =

M =

v = $(-1)^s$ M 2^E =

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

C float Decoding Example

$$v = (-1)^s \, M \, 2^E$$
$$E = \texttt{exp} - Bias$$

float: `0xC0A00000`

binary: 1100 0000 1010 0000 0000 0000 0000 0000

1	1000 0001	010 0000 0000 0000 0000 0000
1	8-bits	23-bits

E =

S =

M = 1 .

v = $(-1)^s$ M 2^E =

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

C float Decoding Example

$$v = (-1)^s \, M \, 2^E$$
$$E = \texttt{exp} - Bias$$

$Bias = 2^{k-1} - 1 = 127$

float: `0xC0A00000`

binary: `1100 0000 1010 0000 0000 0000 0000 0000`

1	1000 0001	010 0000 0000 0000 0000 0000
1	8-bits	23-bits

E = `exp` – Bias = 129 – 127 = 2 (decimal)

S = 1 -> negative number

M = 1.`010 0000 0000 0000 0000 0000`

= `1 + 1/4 = 1.25`

$v = (-1)^s \, M \, 2^E = (-1)^1 * 1.25 * 2^2 = -5$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- **Floating point in C**
- Summary

Floating Point in C

- **C Guarantees Two Levels**
  - `float` single precision
  - `double` double precision
- **Conversions/Casting**
  - Casting between `int`, `float`, and `double` changes bit representation
  - `double`/`float` → `int`
    - Truncates fractional part
    - Like rounding toward zero
    - Not defined when out of range or NaN: Generally sets to TMin
  - `int` → `double`
    - Exact conversion, as long as `int` has ≤ 53 bit word size
  - `int` → `float`
    - Will round according to rounding mode

Floating Point Puzzles

- **For each of the following C expressions, either:**
  - Argue that it is true for all argument values
  - Explain why not true

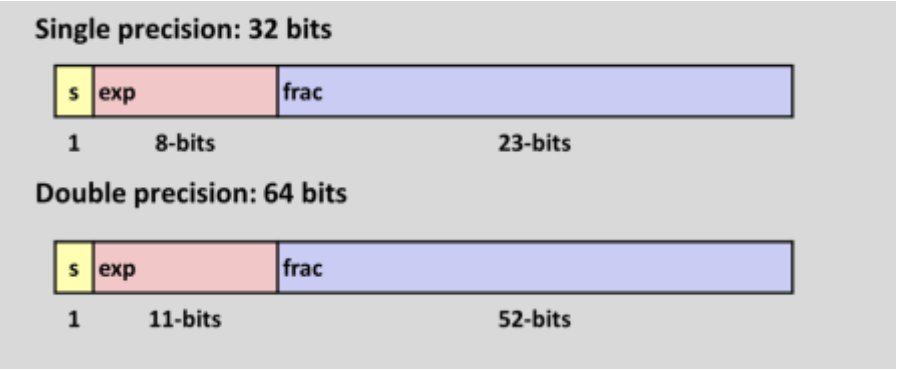
```
int x = …;
float f = …;
double d = …;
```

Assume neither **`d`** nor **`f`** is NaN

- `x == (int)(float) x` ✗
- `x == (int)(double) x` ✓
- `f == (float)(double) f` ✓
- `d == (double)(float) d` ✗
- `f == -(-f);` ✓

C语言中的浮点数类型-32位

- C语言中有float和double类型，分别对应IEEE 754单精度浮点数格式和双精度浮点数格式
- long double类型的长度和格式随编译器和处理器类型的不同而有所不同，IA-32中是80位扩展精度格式
- 从int转换为float时，不会发生溢出，但可能有数据被舍入
- 从int或 float转换为double时，因为double的有效位数更多，故能保留精确值
- 从double转换为float和int时，可能发生溢出，此外，由于有效位数变少，故可能被舍入
- 从float 或double转换为int时，因为int没有小数部分，所以数据可能会向0方向被截断

Summary

- **IEEE Floating Point has clear mathematical properties**
- **Represents numbers of form M x 2^E**
- **One can reason about operations independent of implementation**
  - As if computed with perfect precision and then rounded
- **Not the same as real arithmetic**
  - Violates associativity/distributivity
  - Makes life difficult for compilers & serious numerical applications programmers